

Most of the slides are taken from

Eric Freeman, Elisabeth Robson, Bert Bates,
Kathy Sierra: Head First Design Patterns,
O'Reilly Media 2004



Design Pattern Iterator and Composite



Francesco Guerra
francesco.guerra@unimore.it

Motivating example

```
public class MenuItem {  
    String name;  
    String description;  
    boolean vegetarian;  
    double price;  
  
    public MenuItem(String name,  
                    String description,  
                    boolean vegetarian,  
                    double price)  
    {  
        this.name = name;  
        this.description = description;  
        this.vegetarian = vegetarian;  
        this.price = price;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public String getDescription() {  
        return description;  
    }  
}
```

Let us consider a Restaurant menu.
It has lots of items, each one has a name,
a description, and a price.

```
    public double getPrice() {  
        return price;  
    }  
  
    public boolean isVegetarian() {  
        return vegetarian;  
    }  
}
```

Restaurant1

ArrayList

```
public class Menu1 {
    ArrayList<MenuItem> menuItems;

    public Menu1() {
        menuItems = new ArrayList<MenuItem>();

        addItem("K&B's Pancake Breakfast",
            "Pancakes with scrambled eggs, and toast",
            true,
            2.99);
        ... other items
    }

    public void addItem(String name, String description,
        boolean vegetarian, double price)
    {
        MenuItem menuItem = new MenuItem(name,
description, vegetarian, price);
        menuItems.add(menuItem);
    }

    public ArrayList<MenuItem> getMenuItems() {
        return menuItems;
    }

    // other menu methods here
}
```

Restaurant 2

Array

```
public class Menu2 {
    static final int MAX_ITEMS = 6;
    int numberOfItems = 0;
    MenuItem[] menuItems;

    public Menu2() {
        menuItems = new MenuItem[MAX_ITEMS];

        addItem("Vegetarian BLT",
            "(Fakin') Bacon with lettuce & tomato on whole wheat", true, 2.99);
        // a couple of other Diner Menu items added here
    }

    public void addItem(String name, String description,
        boolean vegetarian, double price)
    {
        MenuItem menuItem = new MenuItem(name, description, vegetarian, price);
        if (numberOfItems >= MAX_ITEMS) {
            System.err.println("Sorry, menu is full!  Can't add item to menu");
        } else {
            menuItems[numberOfItems] = menuItem;
            numberOfItems = numberOfItems + 1;
        }
    }

    public MenuItem[] getMenuItems() {
        return menuItems;
    }
}
```



Merging the menus

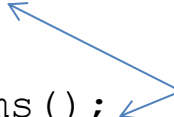
- ▶ What's the problem with having two different menu representations?
- ▶ Let's try implementing a client who uses the two menus.
 - ▶ You have to create a software waiter working according to these specifications
 - ▶ `printMenu()` -prints every item on the menu
 - ▶ `printMenu1()` - prints just menu1 items
 - ▶ `printMenu2()` -prints just menu2 items
 - ▶ `printVegetarianMenu()` - prints all vegetarian menu items
 - ▶ `isItemVegetarian(name)` - given the name of an item, returns true if the item is vegetarian

printMenu()

```
Menu1 menu1= new Menu1();  
ArrayList<MenuItem> menu1Items = menu1.getMenuItems();
```

```
Menu2 menu2 = new Menu2();  
MenuItem[] menu2Items = menu2.getMenuItems();
```

```
for (int i = 0; i < menu1.size(); i++) {  
    MenuItem menuItem = menu1Items.get(i);  
    System.out.print(menuItem.getName() + " ");  
    System.out.println(menuItem.getPrice() + " ");  
    System.out.println(menuItem.getDescription());  
}  
for (int i = 0; i < menu2.length; i++) {  
    MenuItem menuItem = menu2Items[i];  
    System.out.print(menuItem.getName() + " ");  
    System.out.println(menuItem.getPrice() + " ");  
    System.out.println(menuItem.getDescription());  
}
```

 The getMenuItems() method looks the same, but the calls are returning different types.

Implementing every other method is going to be a variation of this theme



Issues

1. We are coding to the Menu1 and Menu2 concrete implementations, not to an interface
2. If we decide to switch to another type of menu that implement the list of menu items with a HashTable, we have to modify the code in Manager
3. The Manager needs to know how each menu represents its internal collection of menu items (this violates encapsulation!)
4. We have duplicated code

Using for ... each to loop

- ▶ It would really be nice if we could implement the same interface for the menus
 - ▶ We minimize the concrete references in the code and also get rid of the multiple loops required to iterate over both menus.
- ▶ Using **for ... each** would allow us to hide the complexity of the different kinds of iteration.
 - ▶ But that doesn't solve the real problem here:
 - ▶ two different implementations of the menus

Using for ... each to loop (2)

```
Menu1 menu1 = new Menu1();
ArrayList<MenuItem> menu1Items= menu1.getMenuItems();
Menu2 menu2 = new menu2();
MenuItem[] menu2Items= menu2.getMenuItems();

for (MenuItem menuItem : menu1Items ) {
    System.out.print(menuItem.getName());
    System.out.println("\t\t" + menuItem.getPrice());
    System.out.println("\t" + menuItem.getDescription());
}
for (MenuItem menuItem : menu2Items ) {
    System.out.print(menuItem.getName());
    System.out.println("\t\t" + menuItem.getPrice());
    System.out.println("\t" + menuItem.getDescription());
}
```

- ▶ Our goal is to decouple the software from the concrete implementations of the menus completely.

Encapsulating the iteration

- ▶ To iterate through the menu1 items we use the size() and get() methods on the ArrayList:

```
for (int i = 0; i < breakfastItems.size(); i++) {  
    MenuItem menuItem = breakfastItems.get(i);  
}
```

- ▶ To iterate through the menu2 items we use we use the Array length field and the array subscript notation on the MenuItem Array:

```
for (int i = 0; i < lunchItems.length; i++) {  
    MenuItem menuItem = lunchItems[i];  
}
```

- ▶ What if we create an object, let's call it an Iterator, that encapsulates the way we iterate through a collection of objects?

```
Iterator iterator = menu1.createIterator();  
while (iterator.hasNext()) {  
    MenuItem menuItem = iterator.next();  
}
```

Both menus implement the same code!

The Iterator Pattern



- ▶ The iterator allows you to hide the internal implementation of the data structure.
- ▶ The Iterator Pattern relies on an interface called `Iterator`.
 - ▶ The `hasNext()` method tells us if there are more elements to iterate.
 - ▶ The `next()` method returns the next object in the aggregate.
- ▶ we can implement `Iterators` for any kind of collection of objects: arrays, lists, hashmaps, ...

Adding an Iterator to Menu

```
public interface Iterator {
    boolean hasNext();
    Object next();
}

public class Menu2Iterator implements Iterator {
    MenuItem[] items;
    int position = 0;

    public Menu2Iterator(MenuItem[] items) {
        this.items = items;
    }

    public MenuItem next() {
        MenuItem menuItem = items[position];
        position = position + 1;
        return menuItem;
    }

    public boolean hasNext() {
        if (position >= items.length || items[position] == null) {
            return false;
        } else {
            return true;
        }
    }
}
```



Refactoring Menu2 with Iterator

```
public class Menu2 {  
    static final int MAX_ITEMS = 6;  
    int numberOfItems = 0;  
    MenuItem[] menuItems;  
  
    // constructor here  
  
    // addItem here  
  
    public MenuItem[] getMenuItems() {  
        return menuItems;  
    }  
  
    public Iterator createIterator() {  
        return new Menu2Iterator(menuItems);  
    }  
  
    // other menu methods here  
}
```

We don't want it , because it exposes our internal implementation

- ▶ We're returning the Iterator interface.
- ▶ The client doesn't need to know how the menuItems are maintained in the Menu, nor does it need to know how the Menu2Iterator is implemented.
- ▶ It just needs to use the iterators to step through the items in the menu.

Printing the Menus

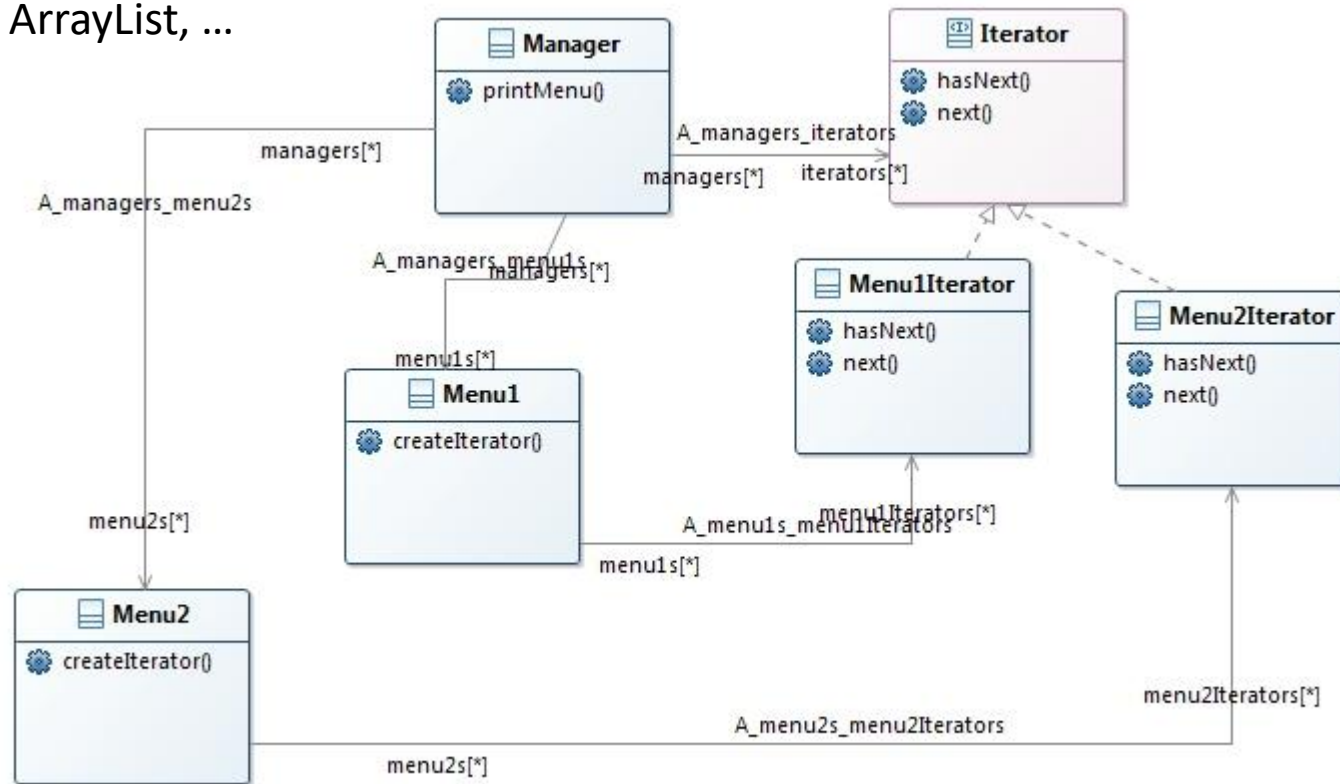
```
public void printMenu() {
    Iterator menu1Iterator = menu1.createIterator();
    Iterator menu2Iterator = menu2.createIterator();
    printMenu(menu1Iterator);
    printMenu(menu2Iterator);
}

private void printMenu(Iterator iterator) {
    while (iterator.hasNext()) {
        MenuItem menuItem = iterator.next();
        System.out.print(menuItem.getName() + ", ");
        System.out.print(menuItem.getPrice() + " -- ");
        System.out.println(menuItem.getDescription());
    }
}
```

Note that the slides do not report the implementation of menu1Iterator

The class representation

The Manager doesn't need to know if a Menu is implemented with an Array, an ArrayList, ...



These two menus implement the same exact set of methods, but they aren't implementing the same interface.

The iterators implement the new `createIterator()` method; they are responsible for creating the iterator for their respective menu items' implementations.

Problems solved and issues

Without iterator

- ▶ The menus are not well encapsulated, one using an ArrayList, the other an Array
- ▶ We need two loops to iterate through the items
- ▶ The printMenu is bound to concrete classes

With iterator

- ▶ The Menu implementations are now encapsulated. printMenu has no idea now how the Menus hold their collection of menu items
- ▶ All we need is a loop that polymorphically handles any collection of items implementing the iterator
- ▶ Now there is an interface

Making improvements using the Java Iterator Interface



- ▶ The Java Iterator interface looks just similar to the previous definition.
 - ▶ There is an additional method that allows us to remove the last item returned by the `next()` method from the aggregate.
 - ▶ The `remove()` method is considered optional.
 - ▶ You don't have to provide remove functionality.
 - ▶ But, you should provide the method because it's part of the Iterator interface.
 - ▶ If you're not going to allow `remove()` in your iterator you'll want to throw the runtime exception `java.lang.UnsupportedOperationException`.
 - ▶ This exception may be thrown from `remove()` and any client that is a good citizen will check for this exception when calling the `remove()` method.

Making improvements using the Java Iterator Interface

- ▶ Not only does `java.util` have its own `Iterator` interface,
 - ▶ `ArrayList` has an `iterator()` method that returns an iterator.
 - ▶ In other words, we never needed to implement our own iterator for `ArrayList`.
- ▶ However, we'll still need our implementation for `Menu2` because it relies on an `Array`
 - ▶ `Array` doesn't support the `iterator()` method (or any other way to create an array iterator).

Menu 1

- ▶ Menu1 is based on ArrayList
 - ▶ delete the Menu1Iterator class,
 - ▶ add an import java.util.Iterator to the top of Menu1
 - ▶ change one line of the Menu1:

```
public Iterator<MenuItem> createIterator() {  
    return menuItems.iterator();  
}
```

← Instead of creating our own iterator now, we just call the iterator() method on the menuItems ArrayList.

► Menu2 implementation (Array)

Using java.util.Iterator

```
import java.util.Iterator;

public class Menu2Iterator implements Iterator {
    MenuItem[] list;
    int position = 0;

    public Menu2Iterator (MenuItem[] list) {this.list = list;}

    public MenuItem next() {
        //implementation here
    }
    public boolean hasNext() {
        //implementation here
    }

    public void remove() {
        if (position <= 0) {
            throw new IllegalStateException
("You can't remove an item until you've done at least one next()");
        }
        if (list[position-1] != null) {
            for (int i = position-1; i < (list.length-1); i++) {
                list[i] = list[i+1];
            }
            list[list.length-1] = null;
        }
    }
}
```

We shift all the element up one when
remove() is called

Using java.util.Iterator (2)

```
public interface Menu {  
    public Iterator<MenuItem> createIterator();  
}
```

```
public class Menu2 implements Menu{  
    static final int MAX_ITEMS = 6;  
    int numberOfItems = 0;  
    MenuItem[] menuItems;  
  
    // constructor here  
  
    // addItem here  
  
    public Iterator createIterator() {  
        return new Menu2Iterator(menuItems);  
    }  
  
}
```

```
The same for Menu1    public Iterator<MenuItem> createIterator() {  
                        return menuItems.iterator();
```

Using java.util.Iterator (3)

```
import java.util.Iterator;

public class Manager {
    Menu Menu1;
    Menu Menu2;

    public Manager(Menu menu1, Menu menu2) {
        this.menu1 = menu1;
        this.menu2 = menu2;
    }

    public void printMenu() {
        Iterator<MenuItem> menu1Iterator = menu1.createIterator();
        Iterator<MenuItem> menu2Iterator = menu2.createIterator();
        printMenu(menu1Iterator);
        printMenu(menu2Iterator);
    }

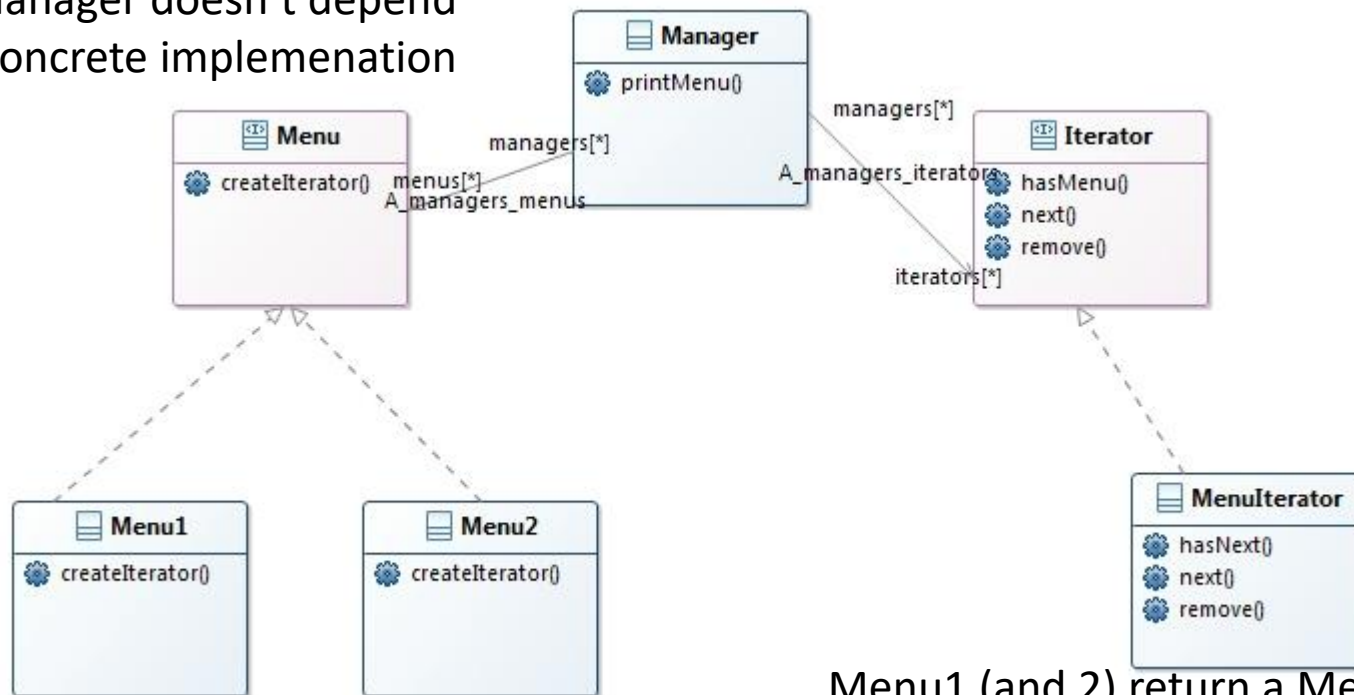
    private void printMenu(Iterator iterator) {
        while (iterator.hasNext()) {
            MenuItem menuItem = (MenuItem)iterator.next();
            System.out.print(menuItem.getName() + ", ");
            System.out.print(menuItem.getPrice() + " -- ");
            System.out.println(menuItem.getDescription());
        }
    }
}
```



What is changed?

Manager is decoupled from the implementation of the menus: we can use an Iterator without having to know about how the list of items is implemented

The manager doesn't depend on a concrete implementation



Both the menus now implement the Menu interface, which means they need to implement the new `createIterator()` method.

Menu1 (and 2) return a MenuItem from its `createIterator()` method because that's the kind of iterator required to iterate over its Array of menu items.

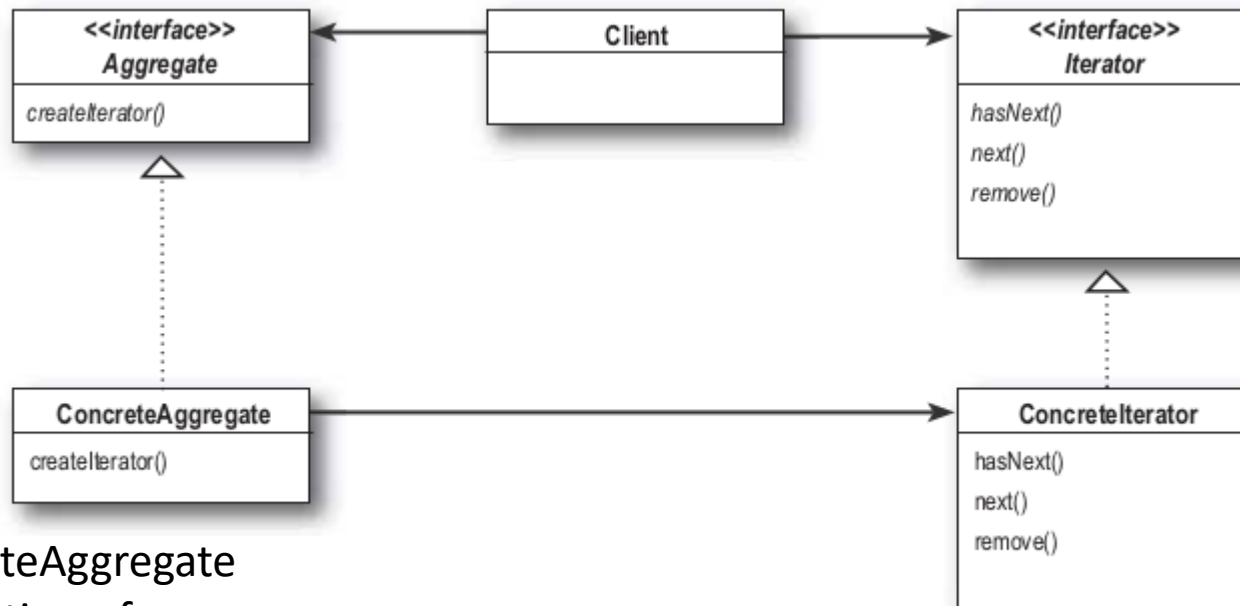
Iterator Pattern defined

- ▶ The Iterator Pattern provides a way to access the elements of an aggregate object sequentially without exposing its underlying representation.
- ▶ The Iterator Pattern allows traversal of the elements of an aggregate without exposing the underlying implementation.
- ▶ It also places the task of traversal on the iterator object, not on the aggregate, which simplifies the aggregate interface and implementation, and places the responsibility where it should be.

Iterator Pattern defined (2)

It decouples the client from the implementation of the collection of objects.

The Iterator interface provides a set of methods for traversing over elements of a collection.



The **ConcreteAggregate** has a collection of objects and implements the method that returns an **Iterator** for its collection.

Each **ConcreteAggregate** is responsible for instantiating a **ConcreteIterator** that can iterate over its collection of objects.

Comments (1)

- ▶ Other books show the Iterator class diagram with the methods `first()`, `next()`, `isDone()` and `currentItem()`.
 - ▶ Those are the “classic” method names that have been used. These names have changed over time and we now have `next()`, `hasNext()` and even `remove()` in `java.util.Iterator`.
 - ▶ Let’s look at the classic methods. The `next()` and `currentItem()` have been merged into one method in `java.util`. The `isDone()` method has obviously become `hasNext()`; but we have no method corresponding to `first()`.
 - ▶ The `remove()` method is an example of an extension in `java.util.Iterator`.

Comments (2)

- ▶ Could I implement an Iterator that can go backwards as well as forwards?
 - ▶ You have to add two methods, one to get to the previous element, and one to tell you when you're at the beginning of the collection of elements.
 - ▶ Java's Collection Framework provides another type of iterator interface called ListIterator. This iterator adds previous() and a few other methods to the standard Iterator interface. It is supported by any Collection that implements the List interface.
- ▶ Who defines the ordering of the iteration in a collection like Hashtable, which are inherently unordered?
 - ▶ Iterators imply no ordering. The underlying collections may be unordered as in a hashtable or in a bag; they may even contain duplicates.
 - ▶ In general, you should make no assumptions about ordering unless the Collection documentation indicates otherwise.

Comments (3)

- ▶ What does it mean that we can write “polymorphic code” using an iterator?
 - ▶ When we write methods that take Iterators as parameters, we are using polymorphic iteration.
 - ▶ That means we are creating code that can iterate over any collection as long as it supports Iterator. We don’t care about how the collection is implemented, we can still write code to iterate over it.
- ▶ Can I use my own iterator implementations with classes that are already using the Java iterators?
 - ▶ If you have a common Iterator interface, it will certainly make it easier for you to mix and match your own aggregates with Java aggregates like ArrayList and Vector.
 - ▶ But remember, if you need to add functionality to your Iterator interface for your aggregates, you can always extend the Iterator interface.

Single Responsibility

- ▶ **Design Principle:**
A class should have only one reason to change.
 - ▶ Every responsibility of a class is an area of potential change.
 - ▶ More than one responsibility means more than one area of change.
- ▶ What if we allowed our aggregates to implement their internal collections and related operations AND the iteration methods?
 - ▶ The can change if the collection changes in some way, and if the way we iterate changes.
- ▶ Cohesion is the term used as a measure of how closely a class or a module supports a single purpose or responsibility.
 - ▶ Cohesion is a more general concept than the Single Responsibility Principle
 - ▶ Classes that adhere to the principle tend to have high cohesion and are more maintainable than classes that take on multiple responsibilities and have low cohesion.

Java Iterators and Collections

- ▶ Java gives you a lot of “collection” classes that allow you to store and retrieve groups of objects.
 - ▶ For example, `Vector` and `LinkedList`.

<i>Collection</i>
+<code>add(element : Object) : boolean</code> +<code>addAll(collection : Collection) : boolean</code> +<code>clear() : void</code> +<code>contains(element : Object) : boolean</code> +<code>containsAll(collection : Collection) : boolean</code> +<code>equals(object : Object) : boolean</code> +<code>hashCode() : int</code> +<code>iterator() : Iterator</code> +<code>remove(element : Object) : boolean</code> +<code>removeAll(collection : Collection) : boolean</code> +<code>retainAll(collection : Collection) : boolean</code> +<code>size() : int</code> +<code>toArray() : Object[]</code> +<code>toArray(array : Object[]) : Object[]</code>

The nice thing about Collections and Iterator is that each Collection object knows how to create its own Iterator.

Adding another menu

```
public class Menu3 {
    HashMap<String, MenuItem> menuItems = new HashMap<String, MenuItem>();

    public Menu3() {
        addItem("Veggie Burger and Air Fries","Veggie burger on a whole
wheat bun, lettuce, tomato, and fries",true, 3.99);
        addItem("Soup of the day","A cup of the soup of the day, with a side
salad",false, 3.69);
        addItem("Burrito","A large burrito, with whole pinto beans, salsa,
guacamole",true, 4.29);
    }

    public void addItem(String name, String description,
                        boolean vegetarian, double price)
    {
        MenuItem menuItem = new MenuItem(name, description, vegetarian, price);
        menuItems.put(menuItem.getName(), menuItem);
    }

    public Map<String, MenuItem> getItems() {
        return menuItems;
    }
}
```

Adapting the menu to our framework

```
public class Menu3 implements Menu {
    HashMap<String, MenuItem> menuItems = new HashMap<String, MenuItem>();

    public Menu3() {
        // constructor code here
    }

    public void addItem(String name, String description,
                        boolean vegetarian, double price)
    {
        MenuItem menuItem = new MenuItem(name, description, vegetarian, price);
        menuItems.put(menuItem.getName(), menuItem);
    }

    public Map<String, MenuItem> getItems() {
        return menuItems;
    +
    public Iterator<MenuItem> createIterator() {
        return menuItems.values().iterator();
    }
}
```

This is an iterator on the values of the HashMap

Adding Menu3 to the Manager

```
public class Manager{
    Menu menu1;
    Menu menu2;
    Menu menu3;

    public Manager(Menu menu1, Menu menu2, Menu menu3) {
        this.menu1 = menu1;
        this.menu2 = menu2;
        this.menu3 = menu3;
    }

    public void printMenu() {
        Iterator<MenuItem> menu1 = menu1.createIterator();
        Iterator<MenuItem> menu2= menu2.createIterator();
        Iterator<MenuItem> menu3 = menu3.createIterator();
        printMenu(menu1);
        printMenu(menu2);
        printMenu(menu3);
    }
}
```

Violating the “Open Closed Principle”?

- ▶ Every time we add a new menu we are going to have to open up the Manager implementation and add more code.
- ▶ The previous implementation
 - ▶ decoupled the menu implementation
 - ▶ extracted the iteration into an iterator.
- ▶ The menus are handled with separate, independent objects
- ▶ Solution
 - ▶ package the menus up into an ArrayList and then
 - ▶ get its iterator to iterate through each Menu.

Solution

```
public class Manager{
    ArrayList<Menu> menus;

    public Manager(ArrayList<Menu> menus) {
        this.menus = menus;
    }

    public void printMenu() {
        Iterator<Menu> menuIterator = menus.iterator();
        while(menuIterator.hasNext()) {
            Menu menu = menuIterator.next();
            printMenu(menu.createIterator());
        }
    }

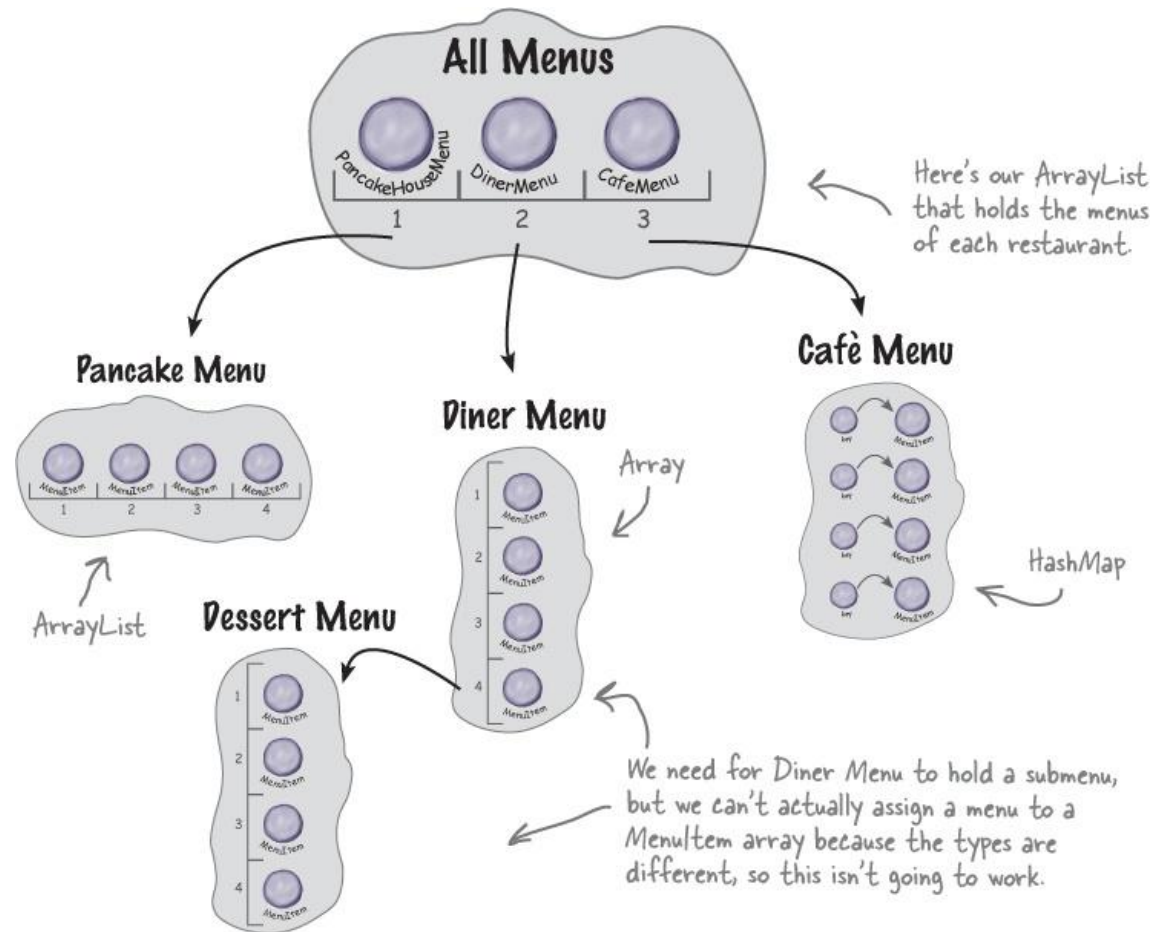
    void printMenu(Iterator<MenuItem> iterator) {
        while (iterator.hasNext()) {
            MenuItem menuItem = iterator.next();
            System.out.print(menuItem.getName() + ", ");
            System.out.print(menuItem.getPrice() + " -- ");
            System.out.println(menuItem.getDescription());
        }
    }
}
```



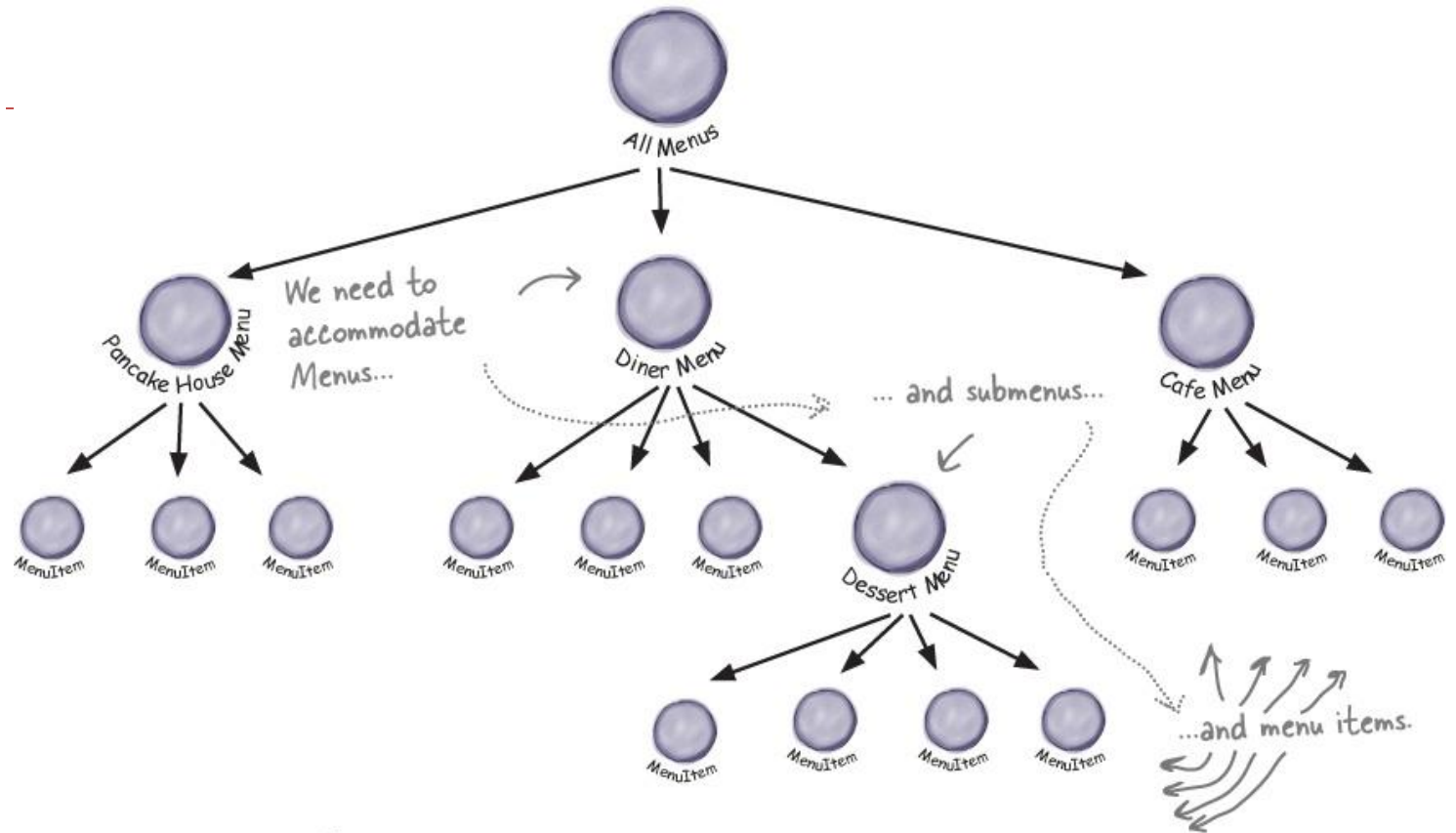
And ... if we want to manage sub-menus?

- ▶ We have rework the implementation into something that is general enough to work over all the menus (and now submenus).
- ▶ We have reached a level of complexity such that if we don't rework the design now, we're never going to have a design that can accommodate further acquisitions or submenus.
 - ▶ We need some kind of a tree-shaped structure that will accommodate menus, submenus, and menu items.
 - ▶ We need to make sure we maintain a way to traverse the items in each menu that is at least as convenient as what we are doing now with iterators.
 - ▶ We may need to traverse the items in a more flexible manner.
 - ▶ For instance, we might need to iterate over only one menu,
 - ▶ or we might need to iterate over an entire menu, including the submenus.
- ▶ Because we need to represent menus, nested submenus and menu items, we can naturally fit them in a tree-like structure.

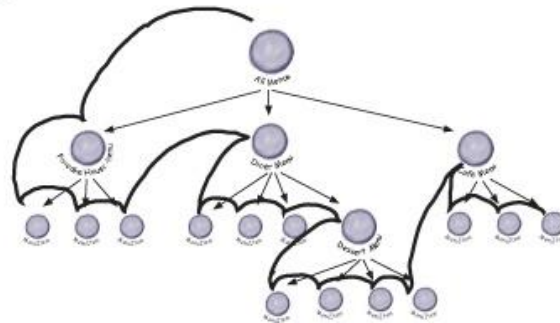
The problem



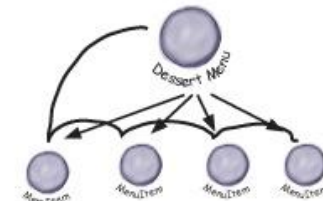
We need a tree-like structure



We still need to be able to traverse all the items in the tree.



We also need to be able to traverse more flexibly, for instance over one menu.



The Composite Pattern defined

- ▶ The Composite Pattern allows you to compose objects into tree structures to represent part-whole hierarchies. Composite lets clients **treat individual objects and compositions of objects uniformly**
- ▶ The Composite Pattern allows us to build structures of objects in the **form of trees that contain both compositions of objects and individual objects as nodes.**
- ▶ Using a composite structure, we can apply the **same operations over both composites and individual objects.**
 - ▶ In other words, in most cases we can ignore the differences between compositions of objects and individual objects.

UML Diagram

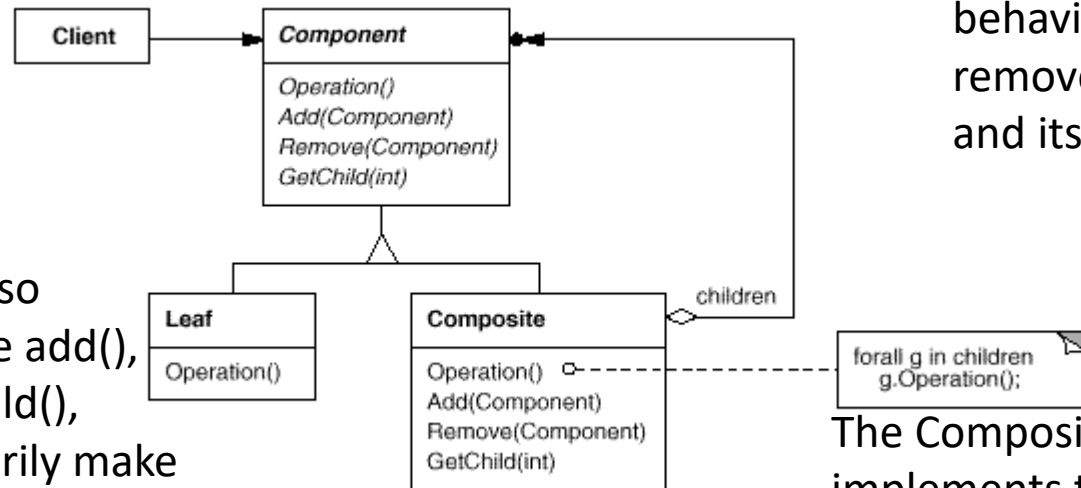
The Client uses the Component interface to manipulate the objects in the composition.

The Component defines an interface for all objects in the composition: both the composite and the leaf nodes.

The Component may implement a default behavior for add(), remove(), getChild() and its operations.

Note that the leaf also inherits methods like add(), remove() and getChild(), which don't necessarily make a lot of sense for a leaf node.

The structure is recursive!

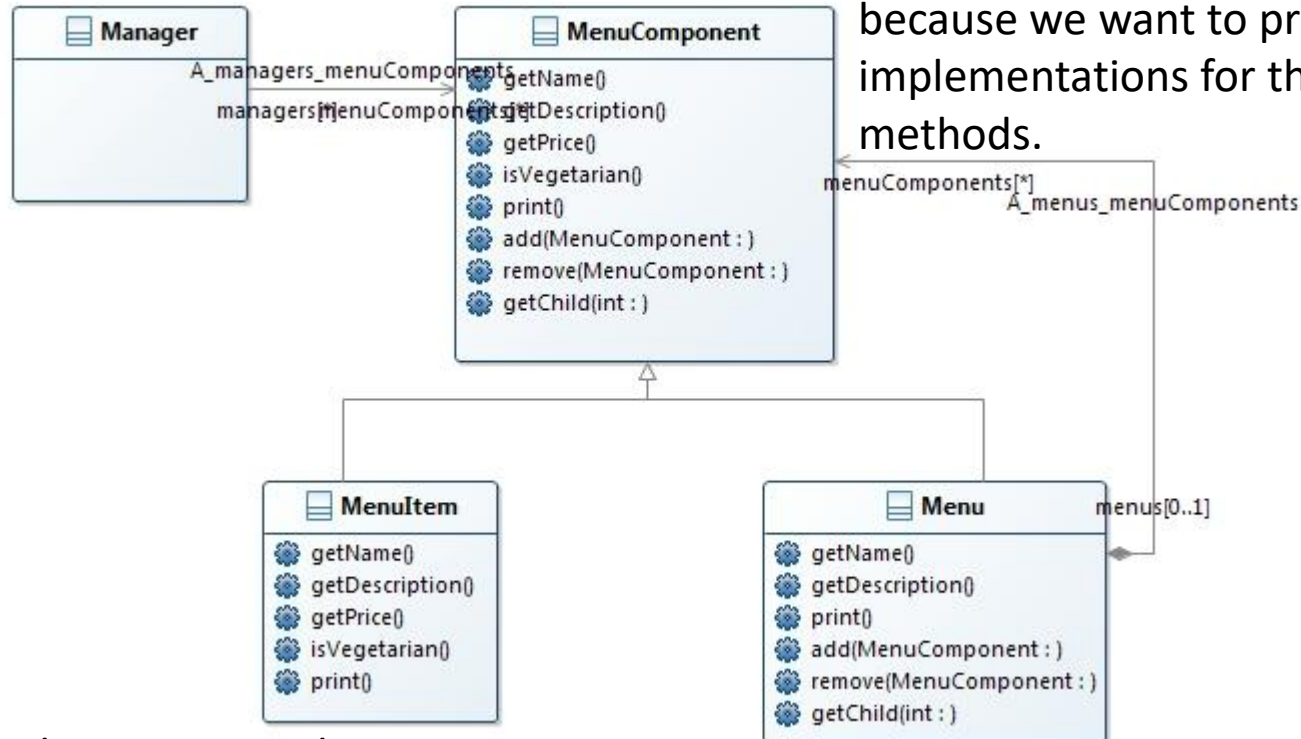


A leaf defines the behavior for the elements in the composition. It does this by implementing the operations the Composite supports.

The Composite also implements the Leaf-related operations. Note that some of these may not make sense on a Composite, so in that case an exception might be generated

Designing Menus with Composite

The Manager uses the MenuComponent interface to access both Menus and MenuItems.



MenuComponent represents the interface for both MenuItem and Menu. It is an abstract class because we want to provide default implementations for these methods.

MenuItem and Menu overrides some methods, and uses the default implementations in other ones

Both MenuItem and Menu override print().

Implementing the Menu Component

- ▶ All components must implement the MenuComponent interface
 - ▶ Leaves and nodes have different roles and it is not possible to define a default implementation for each method that makes sense.
- ▶ Typically the default implementation is UnsupportedOperationException, because of some of the methods only make sense for MenuItem, other for Menu.
 - ▶ if MenuItem or Menu doesn't support an operation, they don't have to do anything; they can just inherit the default implementation.

Implementing the Menu Component

```
public abstract class MenuComponent {  
  
    public void add(MenuComponent menuComponent) {  
        throw new UnsupportedOperationException();  
    }  
    public void remove(MenuComponent menuComponent) {  
        throw new UnsupportedOperationException();  
    }  
    public MenuComponent getChild(int i) {  
        throw new UnsupportedOperationException();  
    }  
    public String getName() {  
        throw new UnsupportedOperationException();  
    }  
    public String getDescription() {  
        throw new UnsupportedOperationException();  
    }  
    public double getPrice() {  
        throw new UnsupportedOperationException();  
    }  
    public boolean isVegetarian() {  
        throw new UnsupportedOperationException();  
    }  
    public void print() {  
        throw new UnsupportedOperationException();  
    }  
}
```

```
public class MenuItem extends MenuComponent {
    String name;
    String description;
    boolean vegetarian;
    double price;
```

Implementing MenuItem

```
    public MenuItem(String name,
                    String description,
                    boolean vegetarian,
                    double price)
    {
        this.name = name;
        this.description = description;
        this.vegetarian = vegetarian;
        this.price = price;
    }

    public String getName() {
        return name;
    }

    public String getDescription() {
        return description;
    }

    public void print() {
        System.out.print("  " + getName());
        if (isVegetarian()) {
            System.out.print("(v)");
        }
        System.out.println(", " + getPrice());
        System.out.println("    -- " + getDescription());
    }

    public double getPrice() {
        return price;
    }

    public boolean isVegetarian() {
        return vegetarian;
    }
}
```

Implementing the Composite Menu

```
public class Menu extends MenuComponent {
    ArrayList<MenuComponent> menuComponents = new ArrayList<MenuComponent>();
    String name;
    String description;
    public Menu(String name, String description) {
        this.name = name;
        this.description = description;    }

    public void add(MenuComponent menuComponent) {
        menuComponents.add(menuComponent);    }

    public void remove(MenuComponent menuComponent) {
        menuComponents.remove(menuComponent);
    }

    public MenuComponent getChild(Integer i) {
        return menuComponents.get(i);    }

    public String getName() {
        return name;    }

    public String getDescription() {
        return description;
    }
}
```



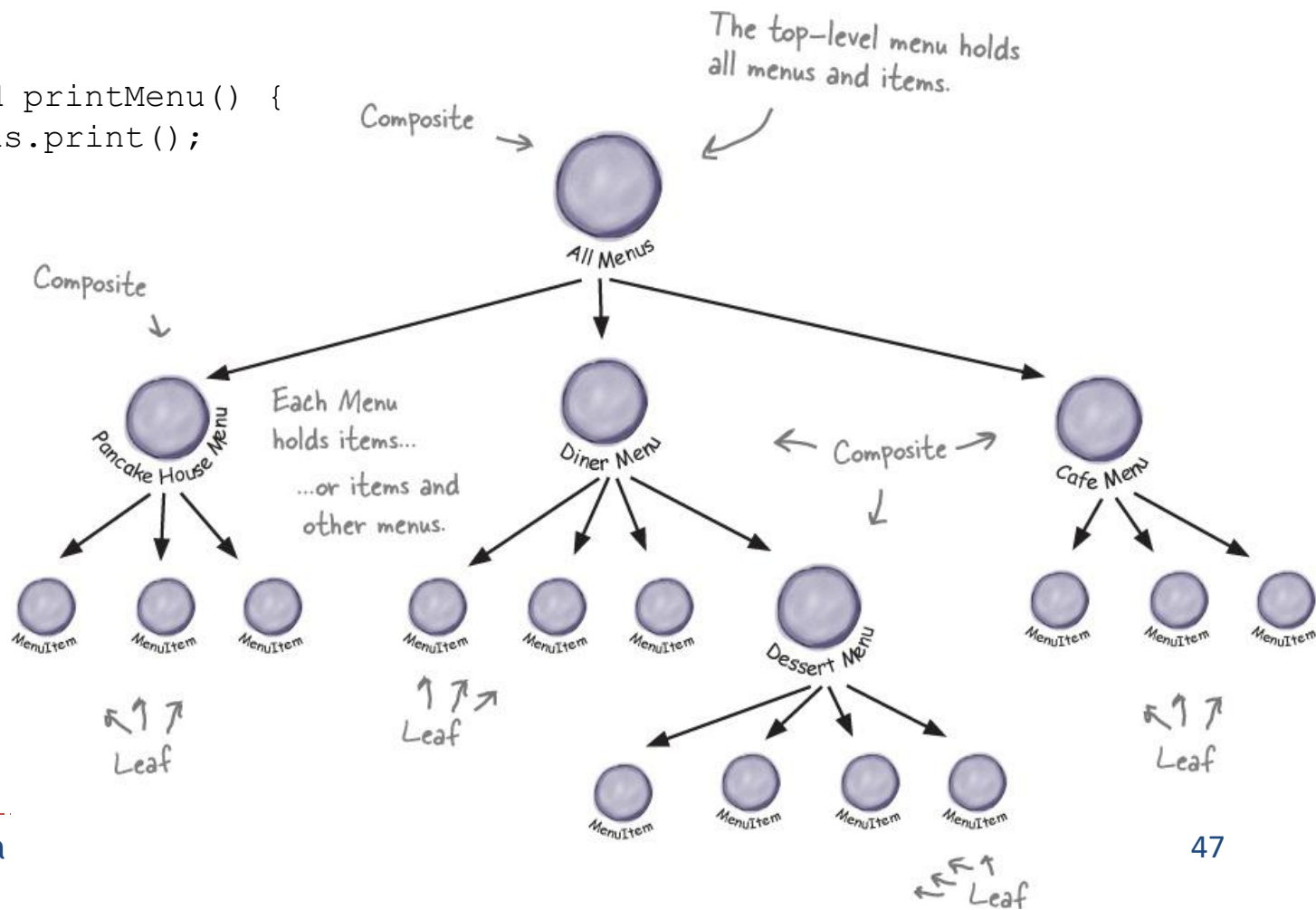
The print Method

```
public void print() {  
    System.out.print("\n" + getName());  
    System.out.println(", " + getDescription());  
    System.out.println("-----");  
  
    Iterator<MenuComponent> iterator = menuComponents.iterator();  
    while (iterator.hasNext()) {  
        MenuComponent menuComponent = iterator.next();  
        menuComponent.print();  
    }  
}
```

- ▶ It should print not only the information about this Menu, but all of this Menu's components: other Menus and MenuItems.
- ▶ We use an Iterator to iterate through all the Menu's components... those could be other Menus, or they could be MenuItems.
- ▶ Since both Menus and MenuItems implement print(), we just call print() and the rest is up to them.

The Manager

```
public class Manager{  
    MenuComponent allMenus;  
  
    public Manager(MenuComponent allMenus) {  
        this.allMenus = allMenus;  
    }  
  
    public void printMenu() {  
        allMenus.print();  
    }  
}
```



The test

```
public class MenuTestDrive {
    public static void main(String args[]) {
        MenuComponent pancakeHouseMenu =
            new Menu("PANCAKE HOUSE MENU", "Breakfast");
        MenuComponent dinerMenu =
            new Menu("DINER MENU", "Lunch");
        MenuComponent cafeMenu =
            new Menu("CAFE MENU", "Dinner");
        MenuComponent dessertMenu =
            new Menu("DESSERT MENU", "Dessert of course!");

        MenuComponent allMenus = new Menu("ALL MENUS", "All menus combined");

        allMenus.add(pancakeHouseMenu);
        allMenus.add(dinerMenu);
        allMenus.add(cafeMenu);

        // add menu items here

        dinerMenu.add(new MenuItem("Pasta", "Spaghetti with Marinara Sauce, and
a slice of sourdough bread", true, 3.89));

        dinerMenu.add(dessertMenu);

        dessertMenu.add(new MenuItem("Apple Pie", "Apple pie with a flakey
crust, topped with vanilla icecream", true, 1.59));
        // add more menu items here

        Manager manager = new Manager(allMenus);
        manager.printMenu();
    }
}
```



One Class, one Responsibility?

- ▶ The Composite Pattern does not follow the rule:
 - ▶ It manages a hierarchy
 - ▶ it performs operations related to Menus.
- ▶ The Composite Pattern takes the Single Responsibility design principle and trades it for transparency.
 - ▶ by allowing the Component interface to contain the child management operations and the leaf operations, a client can treat both composites and leaf nodes uniformly; so whether an element is a composite or leaf node becomes transparent to the client.
- ▶ We lose a bit of safety because a client might try to do something inappropriate or meaningless on an element (like try to add a menu to a menu item).
- ▶ We could take the design in the other direction and separate out the responsibilities into interfaces.
 - ▶ This would make our design safe, but we'd lose transparency and our code would have to use conditionals and the instanceof operator.
- ▶ So, to return to your question, this is a classic case of tradeoff.



Using Iterator with a Composite

```

public class Menu extends MenuComponent {
    ArrayList<MenuComponent> menuComponents = new
        ArrayList<MenuComponent>();
    Iterator<MenuComponent> iterator = null;
    // other code here doesn't change

    public Iterator<MenuComponent> createIterator() {
        if (iterator == null) {
            iterator = new
                CompositeIterator(menuComponents.iterator());
        }
        return iterator;
    }
}

```

CompositeIterator
knows how to
iterate over any
composite.

- ▶ let's add a `createIterator()` method in the abstract `MenuComponent`
 - ▶ calling `createIterator()` on a composite should apply to all children of the composite

The Composite Iterator

```
import java.util.*;
```

```
public class CompositeIterator implements Iterator {
    Stack<Iterator<MenuComponent>> stack = new Stack<Iterator<MenuComponent>>();

    public CompositeIterator(Iterator iterator) {
        stack.push(iterator);
    }

    public MenuComponent next() {
        if (hasNext()) {
            Iterator<MenuComponent> iterator = stack.peek();
            MenuComponent component = iterator.next();
            stack.push(component.createIterator());

            return component;
        } else {
            return null;
        }
    }

    public boolean hasNext() {
        if (stack.empty()) {
            return false;
        } else {
            Iterator<MenuComponent>
            iterator = stack.peek();
            if (iterator == null
                || !iterator.hasNext()) {
                stack.pop();
                return hasNext();
            } else {
                return true;
            }
        }
    }
}
```

If there is a next element, we get the current iterator off the stack and get its next element.

We then throw that component's iterator on the stack. If the component is a Menu, it will iterate over all its items. If the component is a MenuItem, we get the NullIterator, and no iteration happens. Then we return the component.

MenuItem Return null

```
public class MenuItem extends MenuComponent {  
    // other code here doesn't change  
  
    public Iterator<MenuComponent> createIterator() {  
        return null;  
    }  
}
```

- ▶ MenuItem has nothing to iterate over.
- ▶ So how do we handle the implementation of its createIterator() method?
 - ▶ Two choices
- ▶ Choice 1: Return null
 - ▶ We could return null from createIterator(), but then we'd need conditional code in the client to see if null was returned or not.

MenuItem

The Null Iterator

- ▶ Choice 2: Return an iterator that always returns false when hasNext() is called
 - ▶ We can still return an iterator, but the client doesn't have to worry about whether or not null is ever returned. This is an iterator that is a “no op.”

```
public class MenuItem extends MenuComponent {
    // other code here doesn't change

    public Iterator<MenuComponent> createIterator() {
        return new NullIterator();
    }
}

import java.util.Iterator;
public class NullIterator implements Iterator<MenuComponent> {
    public Object next() {
        return null;
    }
    public boolean hasNext() {
        return false;
    }
    public void remove() {
        throw new UnsupportedOperationException();
    }
}
```

Print Vegetarian Menu

```
public class Manager{
    MenuComponent allMenus;

    public Manager(MenuComponent allMenus) {
        this.allMenus = allMenus;
    }

    public void printMenu() {
        allMenus.print();
    }

    public void printVegetarianMenu() {
        Iterator<MenuComponent> iterator = allMenus.createIterator();
        System.out.println("\nVEGETARIAN MENU\n----");
        while (iterator.hasNext()) {
            MenuComponent menuComponent = iterator.next();
            try {
                if (menuComponent.isVegetarian()) {
                    menuComponent.print();
                }
            } catch (UnsupportedOperationException e) {}
        }
    }
}
```

Iterate through every element of the composite.

If an exception happens we catch the exception, but continue with our iteration.



Be careful!

```
try {  
    if (menuComponent.isVegetarian()) {  
        menuComponent.print();  
    }  
} catch (UnsupportedOperationException)  
{}
```

- ▶ In general try/catch is meant for error handling, not program logic.

- ▶ What are our other options?

- ▶ We could have checked the runtime type of the menu component with instanceof to make sure it's a MenuItem before making the call to isVegetarian().
 - But in the process we'd lose transparency because we wouldn't be treating Menus and MenuItems uniformly.
 - We could also change isVegetarian() in the Menu so that it returns false. This provides a simple solution and we keep our transparency.

- ▶ In our solution we are going for clarity: we really want to communicate that this is an unsupported operation on the Menu.

MenuTestDrive

```
public class MenuTestDrive {
    public static void main(String args[]) {

        MenuComponent pancakeHouseMenu =
            new Menu("PANCAKE HOUSE MENU", "Breakfast");
        MenuComponent dinerMenu =
            new Menu("DINER MENU", "Lunch");
        MenuComponent cafeMenu =
            new Menu("CAFE MENU", "Dinner");
        MenuComponent dessertMenu =
            new Menu("DESSERT MENU", "Dessert of course!");

        MenuComponent allMenus = new Menu("ALL MENUS", "All menus combined");

        allMenus.add(pancakeHouseMenu);
        allMenus.add(dinerMenu);
        allMenus.add(cafeMenu);

        pancakeHouseMenu.add(new MenuItem("K&B's Pancake Breakfast",
            "Pancakes with scrambled eggs, and toast", true, 2.99));
        pancakeHouseMenu.add(new MenuItem("Regular Pancake Breakfast",
            "Pancakes with fried eggs, sausage", false, 2.99));
```



MenuTestDrive

```
dinerMenu.add(new MenuItem("Vegetarian BLT","(Fakin') Bacon with lettuce &
tomato on whole wheat", true, 2.99));
dinerMenu.add(new MenuItem("BLT","Bacon with lettuce & tomato on whole
wheat", false, 2.99));
dinerMenu.add(new MenuItem("Pasta","Spaghetti with Marinara Sauce, and a
slice of sourdough bread",true, 3.89));

dinerMenu.add(dessertMenu);

dessertMenu.add(new MenuItem("Apple Pie","Apple pie with a flakey crust,
topped with vanilla icecream",true,1.59));
dessertMenu.add(new MenuItem("Cheesecake","Creamy New York cheesecake",
true,1.99));

cafeMenu.add(new MenuItem("Veggie Burger and Air Fries",
"Veggie burger",true, 3.99));
cafeMenu.add(new MenuItem("Soup of the day","A cup of the soup of the day,
with a side salad",false, 3.69));

Manager manager = new Manager(allMenus);

manager.printVegetarianMenu();

}
```

Key Points

- ▶ An Iterator allows access to an aggregate's elements without exposing its internal structure.
 - ▶ It takes the job of iterating over an aggregate and encapsulates it in another object.
 - ▶ When using an Iterator, we relieve the aggregate of the responsibility of supporting operations for traversing its data.
 - ▶ An Iterator provides a common interface for traversing the items of an aggregate, allowing you to use polymorphism when writing code that makes use of the items of the aggregate.
- ▶ We should strive to assign only one responsibility to each class.
- ▶ The Composite Pattern provides a structure to hold both individual objects and composites.
 - ▶ It allows clients to treat composites and individual objects uniformly.
 - ▶ A Component is any object in a Composite structure. Components may be other composites or leaf nodes.
 - ▶ There are many design tradeoffs in implementing Composite. You need to balance transparency and safety with your needs.

References

- ▶ Eric Freeman, Elisabeth Robson, Bert Bates, Kathy Sierra: Head First Design Patterns, O'Reilly Media 2004
- ▶ Erich Gamma, Ralph Johnson, Richard Helm, John Vlissides: Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley, 1994

MATERIALE DIDATTICO INTERNO DEL CORSO DI Ingegneria del Software.

La diffusione e pubblicazione on line del materiale è assolutamente proibita.